2 - 核心架构设计

本节重点

深入理解企业级项目的架构设计思维，掌握经典设计模式在实际项目中的运用，了解项目初始化过程和后端基础代码。

具体内容包括：

- 设计模式分析
- 核心接口设计
- 工厂模式实战
- 中介者模式实战
- Spring Boot 项目初始化
- 后端基础代码

这一节是整个项目的核心，理解了架构设计思维，后面的开发就会水到渠成，建议大家重点关注设计模式的运用思路。

一、设计模式分析

在开始之前，我们先思考一个问题：如果不使用设计模式，直接把监控逻辑和回答逻辑写在一起会怎样？

假设我们有一个简单的实现：

▼
java
复制代码
```
public class SimpleAutoReply {
    public void run() {
        // 监控知识星球
        List<Question> questions = getZsxqQuestions();
        for (Question question : questions) {
            // 调用 OpenAI 回答
            String answer = callOpenAI(question.getText());
            // 回复到知识星球
            replyToZsxq(question.getId(), answer);
        }
    }
}
```

这种写法很直观，但存在一些问题：

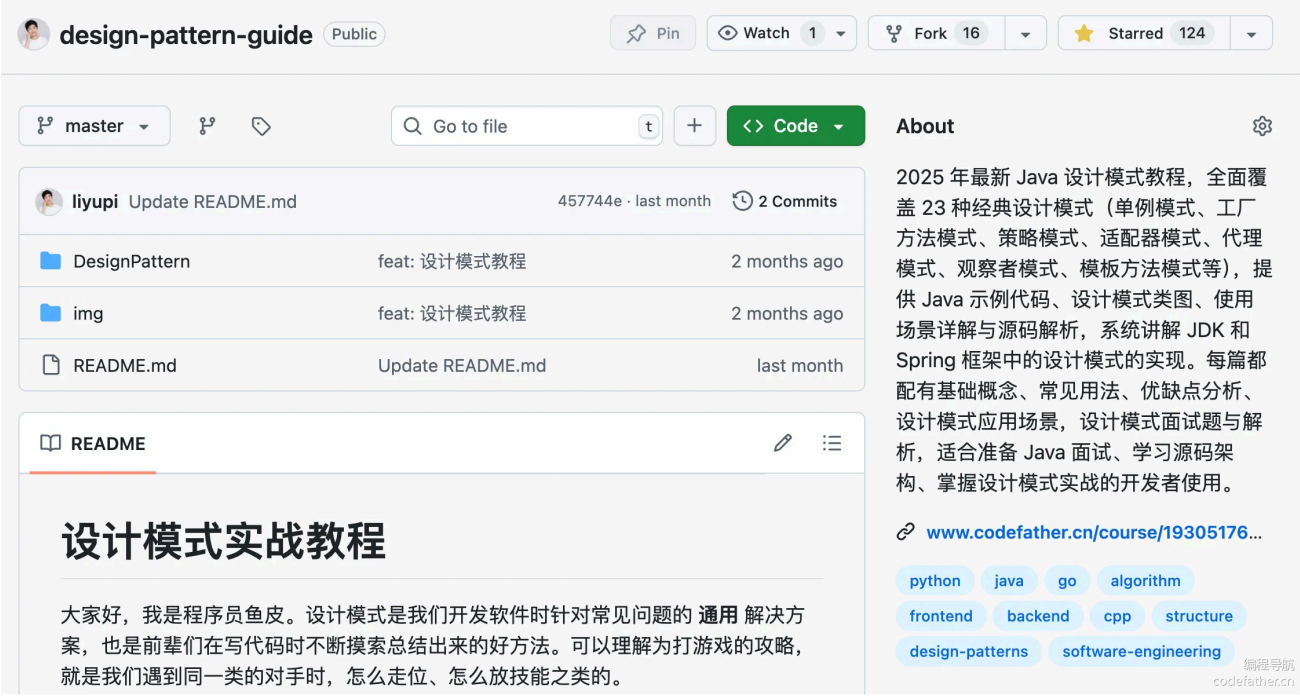
1. 如果要支持微信群监控，需要修改 `getZsxqQuestions()` 方法
2. 如果要换成其他 AI 模型，需要修改 `callOpenAI()` 方法
3. 如果要支持多个任务并发，需要重写整个逻辑

4. 代码耦合度高，难以测试和维护

这就是为什么我们需要设计模式！

设计模式不是为了炫技，而是为了解决实际问题，让代码更加优雅、可维护、可扩展。

💡 如果是第一次接触设计模式，推荐阅读 鱼皮的保姆级设计模式教程



设计模式应用方案

根据我们的需求，项目中主要运用了以下几种设计模式。

1、工厂模式

应用场景：根据配置动态创建不同类型的监控者和回答者

它解决的问题：避免在业务代码中写大量的 if-else 判断来创建对象

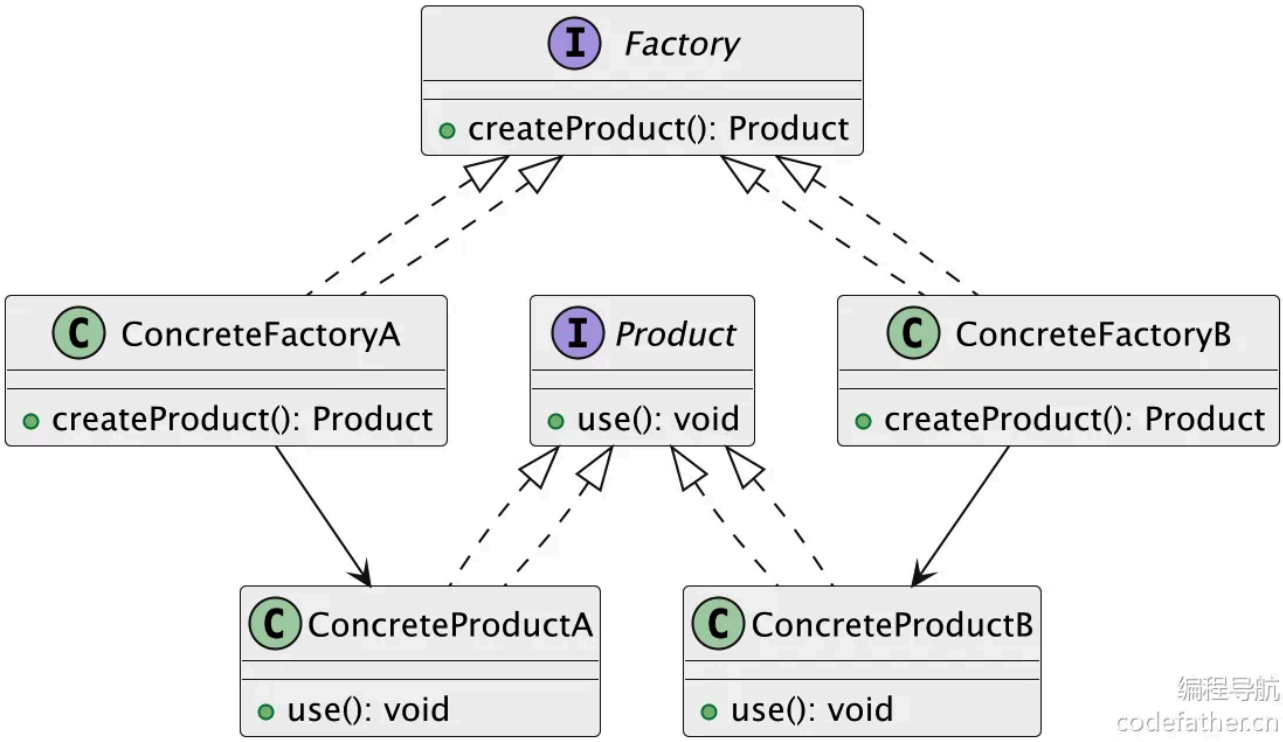
实现思路：

▼

```
java
复制代码
// 不用工厂模式的写法
if ("zsxq".equals(type)) {
    monitor = new ZsxqMonitor();
} else if ("wechat".equals(type)) {
    monitor = new WechatMonitor();
} else {
    monitor = new DefaultMonitor();
}

// 使用工厂模式的写法
Monitor monitor = MonitorFactory.createMonitor(type, config);
```

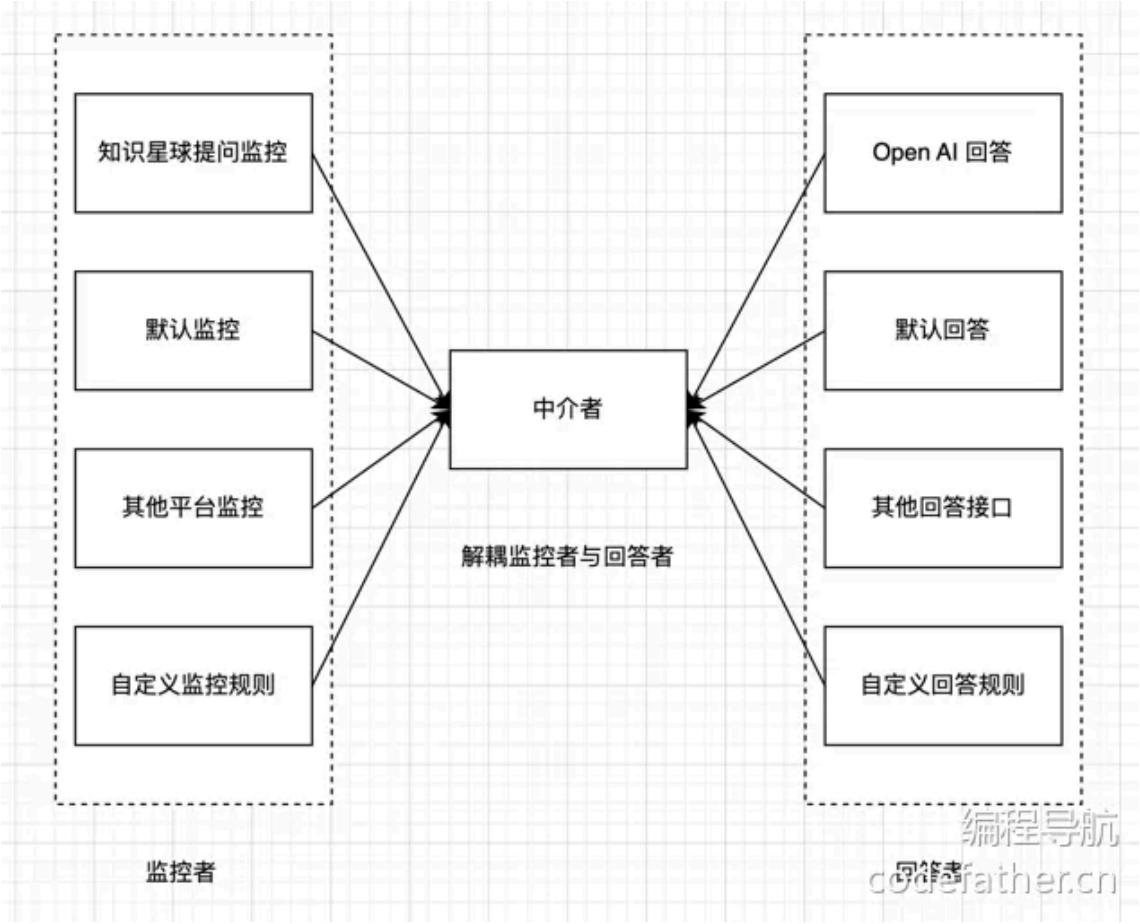
工厂模式的类图：



2、中介者模式

在项目中，中介者模式主要用于协调监控者和回答者的交互。解决了监控者和回答者直接耦合的问题，让它们可以独立变化。

实现思路：不是让 ZsxqMonitor 直接依赖 OpenAiAnswerer，而是通过 JobMediator 来协调它们的交互。

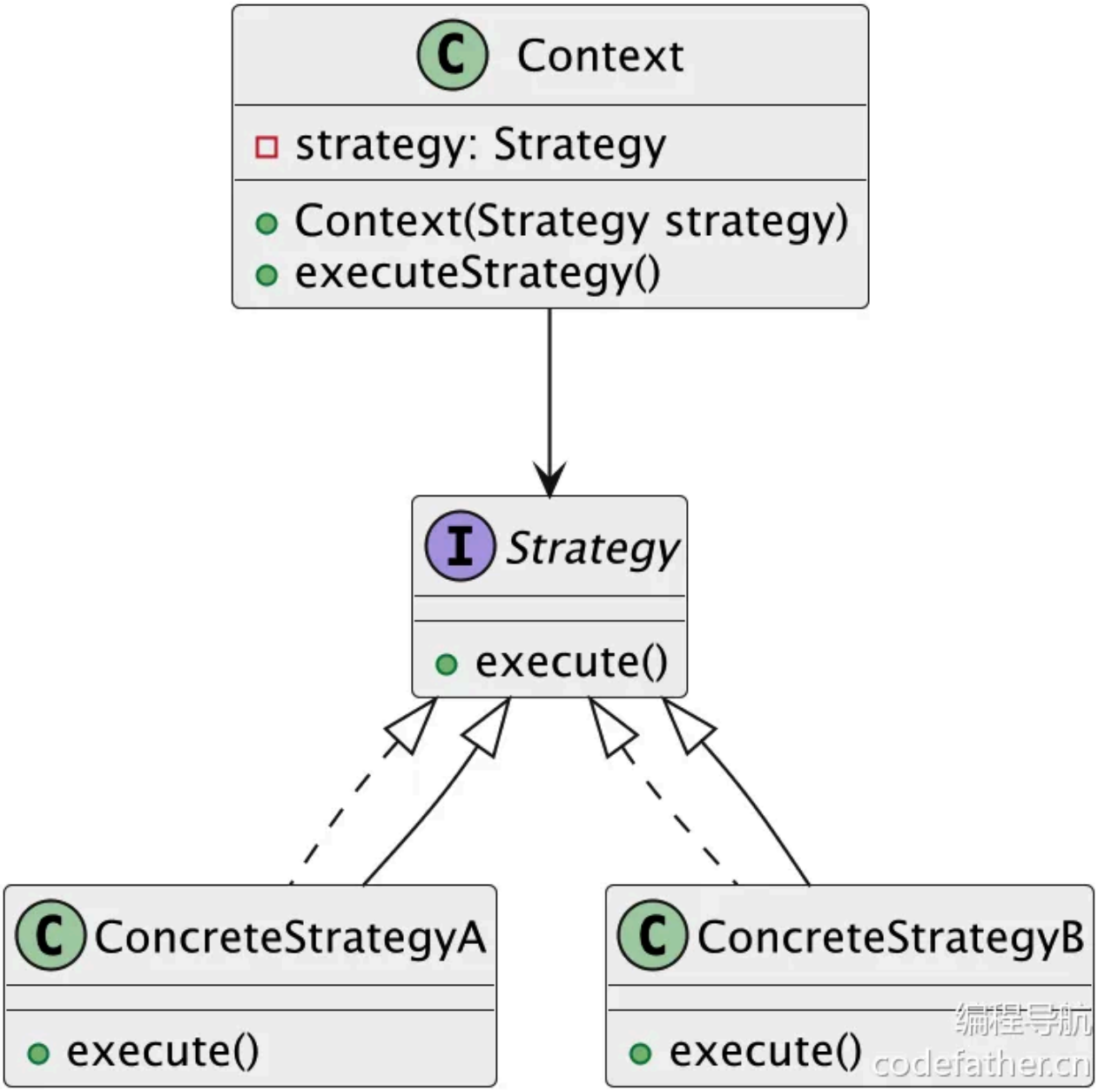


3、策略模式

把不同的监控和回答模式封装为策略，不同的监控策略和回答策略可以动态切换，让算法的变化不影响使用算法的客户端。

实现思路：通过接口抽象，每种监控方式和回答方式都是一种策略。

策略模式的类图比较简单：



架构设计的核心思想

除了设计模式外，整个项目的架构设计遵循了几个重要的原则（其实很多设计模式就是基于这些原则而成的）。

1）单一职责原则：每个类只负责一个功能。Monitor 只负责监控，Answerer 只负责回答，JobMediator 只负责协调。

2）开闭原则：对扩展开放，对修改关闭。要新增一种监控方式，只需要实现 Monitor 接口，不需要修改现有代码。

3）依赖倒置原则：高层模块不依赖低层模块，都依赖于抽象。JobMediator 依赖的是 Monitor 和 Answerer 接口，而不是具体实现。

4）接口隔离原则：使用多个专门的接口，而不是单一的总接口。Monitor 和 Answerer 是两个独立的接口。

这些设计原则看起来很抽象，但在实际编码中你会慢慢发现它们的威力。尤其是当你需要扩展功能时，会发现原来的代码几乎不需要修改，这就是好架构的价值！

二、核心接口设计

监控者接口

首先我们来看项目中监控者的抽象设计。监控者的职责很明确：监控某个平台，发现新消息后通知回答者进行处理。

▼
java
复制代码
```
public abstract class Monitor {

    TaskListItem taskListItem;

    Monitor(TaskListItem taskListItem) {
        this.taskListItem = taskListItem;
    }

    /**
     * 触发监控
     *
     * @param answerer
     */
    public abstract void onMonitor(Answerer answerer);
}
```

这里有几个设计细节值得注意：

1）为什么用抽象类而不是接口？因为所有的监控者都需要任务配置信息，所以用抽象类来提供通用的属性和构造方法。

2）为什么要传入 Answerer 参数？这体现了依赖注入的思想，监控者不需要知道具体用哪个回答者，由外部传入，实现了解耦。

3）TaskListItem 是什么？这是任务配置项，包含了任务名称、监控类型、回答类型、执行周期等信息。

回答者接口

回答者的接口设计相对简单，就是接收一个问题，返回一个答案：

▼
java
复制代码
```
public interface Answerer {

    /**
     * 回答
     * @param prompt 提示语
     * @return 回答结果
     */
    String doAnswer(String prompt);
}
```

这里有 2 个小细节：

1）为什么参数叫 prompt 而不是 question？

因为传给 AI 的不一定是原始问题，可能会加上一些提示词来优化回答效果。

2）为什么返回 String 而不是复杂对象？

遵循 KISS 原则（Keep It Simple, Stupid），简单的设计更容易理解和维护。

任务配置模型

项目中的任务配置模型定义如下：

▼
java
复制代码
```
@Data
public class TaskListItem {

    /**
     * 任务名
     */
    private String name = "";

    /**
     * 任务执行周期
     */
    private String cron = "0/30 * * * * ?";

    /**
     * 回答者
     */
    private String answerer = "openai";

    /**
     * 监控者
     */
    private String monitor = "zsxq";
}
```

这个模型对应配置文件中的任务配置：

▼
yaml
复制代码
```
task:
  list:
    - name: task1
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'
```

通过这种配置驱动的方式，我们可以灵活地定义多个任务，每个任务都可以独立配置监控源和回答源。

三、工厂模式实现

监控者工厂

项目中的监控者工厂实现如下：

▼
java
复制代码
```
public class MonitorFactory {

    /**
```

```
     * 创建监视者
     *
     * @param monitor
     * @param taskListItem
     * @return
     */
    public static Monitor createMonitor(String monitor, TaskListItem taskListItem) {
        switch (monitor) {
            case "zsxq":
                return new ZsxqMonitor(taskListItem);
            default:
                return new DefaultMonitor(taskListItem);
        }
    }
}
```

这里使用了静态工厂方法的形式，简单直接。当需要新增监控者类型时，只需要在 switch 语句中添加一个 case 即可。

学过反射的同学可能会好奇：为什么鱼皮你不用反射啊？

虽然反射可以让代码更加动态，但会增加复杂度，而且编译期无法检查错误。对于这个项目来说，简单的 switch 语句已经足够，可以按需优化。

为什么传入 TaskListItem 呢？

因为不同的监控者可能需要不同的配置信息，比如知识星球监控需要 cookie 和 groupId。

回答者工厂

项目中的回答者工厂实现如下：

▼
java
复制代码

```
public class AnswererFactory {

    /**
     * 创建回答者
     *
     * @param answerer
     * @return
     */
    public static Answerer createAnswerer(String answerer) {
        switch (answerer) {
            case "openai":
                return new OpenAiAnswerer();
            default:
                return new DefaultAnswerer();
        }
    }
}
```

为什么回答者工厂不需要传入配置？

其实这里可以有自己的考虑，鱼皮当时的想法是，回答者的配置（如 API Key）通常是全局的，可以直接通过 Spring 的配置管理来处理。

工厂模式的优势

通过工厂模式，我们的项目获得了这些优势：

1. 解耦创建逻辑：业务代码不需要关心具体创建哪个实现类
2. 集中管理：所有的创建逻辑都集中在工厂类中，便于维护
3. 易于扩展：新增类型只需要修改工厂类，不影响其他代码
4. 统一接口：所有通过工厂创建的对象都遵循同一接口规范

四、中介者模式实现

任务中介者

中介者模式是这个项目最巧妙的设计。JobMediator 作为中介者，协调监控者和回答者的交互：
▼
java
复制代码
```
public class JobMediator implements Runnable {

    private final TaskListItem taskListItem;

    public JobMediator(TaskListItem taskListItem) {
        this.taskListItem = taskListItem;
    }

    @Override
    public void run() {
        // 根据配置选择 monitor 和 answerer
        Monitor monitor = MonitorFactory.createMonitor(taskListItem.getMonitor(), taskListItem);
        Answerer answerer = AnswererFactory.createAnswerer(taskListItem.getAnswerer());
        // 监控并回答
        monitor.onMonitor(answerer);
    }
}
```

为什么中介者类要实现 Runnable 接口呢？

因为每个任务都需要定时执行，实现 Runnable 接口可以方便地与 Spring 的定时任务调度器集成。

通过中介者模式，Monitor 和 Answerer 之间没有直接依赖关系，它们甚至都不知道对方的存在，所有的协调逻辑都在 JobMediator 中。

中介者模式的优势

想象一下，如果没有中介者模式，ZsxqMonitor 直接依赖 OpenAiAnswerer，那么：

- 要换成其他 AI 模型，需要修改 ZsxqMonitor
- 要新增微信群监控，需要重新实现一遍回答逻辑
- 测试时很难 mock 依赖关系

有了中介者模式，这些问题都迎刃而解！

五、项目初始化

这里只是简单介绍项目初始化的过程，实际大家下载鱼皮的源码学习就好。

环境准备

开发环境要求：

- JDK 8 或以上版本
- Maven 3.6 或以上版本
- IDEA 或其他 Java IDE
- Git 版本控制工具

对于这个项目，推荐使用 JDK 8，因为这是企业中使用最广泛的版本，兼容性最好。

创建 Spring Boot 项目

使用 IDEA 的 Spring Initializr 创建项目，选择以下依赖：

- Spring Boot 2.7.2
- Spring Web
- Spring Boot DevTools
- Lombok
- Spring Boot Configuration Processor

主要依赖

项目中集成了以下主要依赖：

1）Hutool 工具库

Hutool 是一个功能丰富的 Java 工具包，提供了 HTTP 请求、JSON 处理、字符串操作等常用功能。

2）Gson JSON 库（可选，也可以直接用 Hutool 的 JSON 工具类）

用于 JSON 数据的序列化和反序列化。

3）Knife4j 接口文档

提供 Swagger 风格的接口文档，方便 API 调试。

4）Apache Commons Lang3（可选）

提供了大量通用的工具类，如字符串操作、数组操作等。

5）Spring Boot AOP

支持面向切面编程，用于实现横切关注点。

项目的 Maven 依赖配置如下：

▼

xml

复制代码

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-aop</artifactId>
    </dependency>
    <!-- https://doc.xiaominfo.com/knife4j/documentation/get_start.html-->
    <dependency>
        <groupId>com.github.xiaoymin</groupId>
        <artifactId>knife4j-spring-boot-starter</artifactId>
        <version>3.0.3</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.apache.commons/commons-lang3 -->
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-lang3</artifactId>
    </dependency>
    <!-- https://mvnrepository.com/artifact/com.google.code.gson/gson -->
    <dependency>
        <groupId>com.google.code.gson</groupId>
        <artifactId>gson</artifactId>
        <version>2.9.1</version>
    </dependency>
    <!-- https://hutool.cn/docs/index.html#/-->
    <dependency>
        <groupId>cn.hutool</groupId>
        <artifactId>hutool-all</artifactId>
        <version>5.8.8</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-configuration-processor</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
```

```
        <scope>test</scope>
    </dependency>
</dependencies>
```

项目结构设计

合理的项目结构是代码可维护性的基础，项目的实际结构如下：
▼
plain
复制代码

```
src/main/java/com/yupi/autoreply/
├── MainApplication.java          // 主启动类
├── answerer/                     // 回答者包
│   ├── Answerer.java             // 回答者接口
│   ├── DefaultAnswerer.java      // 默认回答者
│   └── OpenAiAnswerer.java       // OpenAI 回答者
├── api/                         // 外部 API 包
│   ├── openai/                   // OpenAI API
│   │   ├── model/                // OpenAI 模型
│   │   │   ├── CreateCompletionRequest.java
│   │   │   ├── CreateCompletionResponse.java
│   │   │   └── ModelConstant.java
│   │   └── OpenAiApi.java        // OpenAI API 调用
│   └── zsxq/                     // 知识星球 API
│       ├── model/                // 知识星球模型
│       │   ├── AnswerRequest.java
│       │   ├── AnswerResponse.java
│       │   ├── ListTopicsRequest.java
│       │   └── ListTopicsResponse.java
│       └── ZsxqApi.java          // 知识星球 API 调用
├── common/                      // 通用包
│   ├── BaseResponse.java         // 统一响应格式
│   ├── ErrorCode.java           // 错误码定义
│   └── ResultUtils.java         // 返回结果工具类
├── config/                      // 配置包
│   ├── CorsConfig.java          // 跨域配置
│   ├── Knife4jConfig.java       // 接口文档配置
│   ├── OpenAiConfig.java        // OpenAI 配置
│   ├── TaskConfig.java          // 任务配置
│   └── ZsxqConfig.java          // 知识星球配置
├── controller/                  // 控制器包
│   └── MainController.java      // 主控制器
├── exception/                   // 异常处理包
│   ├── BusinessException.java   // 业务异常
│   ├── GlobalExceptionHandler.java // 全局异常处理器
│   └── ThrowUtils.java          // 抛异常工具类
├── factory/                     // 工厂包
│   ├── AnswererFactory.java     // 回答者工厂
│   └── MonitorFactory.java      // 监控者工厂
├── job/                         // 任务包
│   └── JobMediator.java         // 任务中介者
├── model/                       // 模型包
│   └── TaskListItem.java        // 任务配置项
├── monitor/                     // 监控者包
│   ├── Monitor.java             // 监控者抽象类
│   ├── DefaultMonitor.java      // 默认监控者
│   └── ZsxqMonitor.java         // 知识星球监控者
└── utils/                       // 工具包
    └── SpringContextUtils.java  // Spring 上下文工具
```

这种分包方式的优势：

- 职责清晰：每个包都有明确的职责
- 便于维护：相关功能聚集在一起
- 易于扩展：新增功能时知道应该放在哪个包
- 符合规范：遵循企业级项目的标准结构

六、后端基础代码

配置管理

好的配置管理是项目灵活性的关键。我们的 `application.yml` 配置文件结构如下：

▼
yaml
复制代码
```yaml
spring:
  application:
    name: yu-auto-reply
  mvc:
    pathmatch:
      matching-strategy: ant_path_matcher
server:
  address: 0.0.0.0
  port: 8080
  servlet:
    context-path: /api

# OpenAI 配置
openai:
  model: ${OPENAI_MODEL:text-davinci-003}
  apiKey: ${OPENAI_API_KEY:你的apiKey}

# 知识星球配置
zsxq:
  cookie: ${ZSXQ_COOKIE:你的星球cookie}
  groupId: ${ZSXQ_GROUP_ID:你的星球id}
  silenced: ${ZSXQ_SILENCED:true}

# 任务配置
task:
  concurrent:
    enable: false
    size: 1
  list:
    - name: task1
      monitor: zsxq
      answerer: openai
      cron: '0/30 * * * * ?'
```

这段配置中，值得学习的是环境变量和默认值的设计。使用 `${ENV_VAR:default_value}` 语法支持环境变量覆盖，并且为所有配置项提供合理的默认值。

有了这些配置后，我们需要使用配置类来读取这些配置。

1、OpenAI 配置类

▼
java
复制代码
```java
@Configuration
@ConfigurationProperties(prefix = "openai")
@Data
public class OpenAiConfig {

    /**
     * 模型
     */
    private String model = ModelConstant.TEXT_DAVINCI_003;

    /**
     * apiKey
     */
    private String apiKey;
}
```

2、知识星球配置类

▼
java

复制代码
```
@Configuration
@ConfigurationProperties(prefix = "zsxq")
@Data
public class ZsxqConfig {

    /**
     * 登录 cookie
     */
    private String cookie;

    /**
     * 星球 id
     */
    private String groupId;

    /**
     * 是否提醒提问者
     */
    private Boolean silenced = true;
}
```

3、跨域配置类

▼

java
复制代码
```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        // 覆盖所有请求
        registry.addMapping("/**")
                // 允许发送 Cookie
                .allowCredentials(true)
                // 放行哪些域名（必须用 patterns，否则 * 会和 allowCredentials 冲突）
                .allowedOriginPatterns("*")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .exposedHeaders("*");
    }
}
```

4、接口文档配置类

▼

java
复制代码
```
@Configuration
@EnableSwagger2
@Profile({"dev", "test"})
public class Knife4jConfig {

    @Bean
    public Docket defaultApi2() {
        return new Docket(DocumentationType.SWAGGER_2)
                .apiInfo(new ApiInfoBuilder()
                        .title("接口文档")
                        .description("yu-auto-reply")
                        .version("1.0")
                        .build())
                .select()
                // 指定 Controller 扫描包路径
                .apis(RequestHandlerSelectors.basePackage("com.yupi.autoreply.controller"))
                .paths(PathSelectors.any())
                .build();
    }
}
```

5、任务配置类

任务配置类负责定时任务的注册和调度，详细实现在本教程的第 5 节会介绍。

异常处理机制

项目中实现了完整的异常处理机制：

1、自定义业务异常类

▼
java
复制代码
```
public class BusinessException extends RuntimeException {

    /**
     * 错误码
     */
    private final int code;

    public BusinessException(int code, String message) {
        super(message);
        this.code = code;
    }

    public BusinessException(ErrorCode errorCode) {
        super(errorCode.getMessage());
        this.code = errorCode.getCode();
    }

    public BusinessException(ErrorCode errorCode, String message) {
        super(message);
        this.code = errorCode.getCode();
    }

    public int getCode() {
        return code;
    }
}
```

2、全局异常处理器

利用 `@RestControllerAdvice` 注解实现：

▼
java
复制代码
```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public BaseResponse<?> businessExceptionHandler(BusinessException e) {
        log.error("BusinessException", e);
        return ResultUtils.error(e.getCode(), e.getMessage());
    }

    @ExceptionHandler(RuntimeException.class)
    public BaseResponse<?> runtimeExceptionHandler(RuntimeException e) {
        log.error("RuntimeException", e);
        return ResultUtils.error(ErrorCode.SYSTEM_ERROR, "系统错误");
    }
}
```

3、抛异常工具类

▼
java
复制代码
```
public class ThrowUtils {

    /**
     * 条件成立则抛异常
     */
    public static void throwIf(boolean condition, RuntimeException runtimeException) {
        if (condition) {
            throw runtimeException;
        }
    }
```

```
    /**
     * 条件成立则抛异常
     */
    public static void throwIf(boolean condition, ErrorCode errorCode) {
        throwIf(condition, new BusinessException(errorCode));
    }

    /**
     * 条件成立则抛异常
     */
    public static void throwIf(boolean condition, ErrorCode errorCode, String message) {
        throwIf(condition, new BusinessException(errorCode, message));
    }
}
```

有了这玩意之后，不用每次都自己 throw new 异常了。

通用响应类

项目使用统一的响应格式：

1、基础响应类

▼
java
复制代码

```
@Data
public class BaseResponse<T> implements Serializable {

    private int code;
    private T data;
    private String message;

    public BaseResponse(int code, T data, String message) {
        this.code = code;
        this.data = data;
        this.message = message;
    }

    public BaseResponse(int code, T data) {
        this(code, data, "");
    }

    public BaseResponse(ErrorCode errorCode) {
        this(errorCode.getCode(), null, errorCode.getMessage());
    }
}
```

2、返回结果工具类

▼
java
复制代码

```
public class ResultUtils {

    /**
     * 成功
     */
    public static <T> BaseResponse<T> success(T data) {
        return new BaseResponse<>(0, data, "ok");
    }

    /**
     * 失败
     */
    public static BaseResponse error(ErrorCode errorCode) {
        return new BaseResponse<>(errorCode);
    }

    /**
     * 失败
     */
    public static BaseResponse error(int code, String message) {
        return new BaseResponse(code, null, message);
```

```
    }

    /**
     * 失败
     */
    public static BaseResponse error(ErrorCode errorCode, String message) {
        return new BaseResponse(errorCode.getCode(), null, message);
    }
}
```

Spring 上下文工具类

项目开发中，有时我们需要在非 Spring 管理的类中获取 Spring 的 Bean，可以通过编写一个工具类来实现。

代码如下：

▼
java
复制代码

```
@Component
public class SpringContextUtils implements ApplicationContextAware {

    private static ApplicationContext applicationContext;

    @Override
    public void setApplicationContext(@NotNull ApplicationContext applicationContext) throws BeansException {
        SpringContextUtils.applicationContext = applicationContext;
    }

    /**
     * 通过名称获取 Bean
     *
     * @param beanName
     * @return
     */
    public static Object getBean(String beanName) {
        return applicationContext.getBean(beanName);
    }

    /**
     * 通过 class 获取 Bean
     *
     * @param beanClass
     * @param <T>
     * @return
     */
    public static <T> T getBean(Class<T> beanClass) {
        return applicationContext.getBean(beanClass);
    }

    /**
     * 通过名称和类型获取 Bean
     *
     * @param beanName
     * @param beanClass
     * @param <T>
     * @return
     */
    public static <T> T getBean(String beanName, Class<T> beanClass) {
        return applicationContext.getBean(beanName, beanClass);
    }
}
```

这个工具类在工厂模式中特别有用，因为工厂类通常是静态方法，无法直接注入 Spring Bean。

七、扩展思路

1）实现更多的设计模式，比如观察者模式来处理监控事件，装饰器模式来增强回答功能

2）实现插件化架构，支持通过 SPI 机制动态加载监控者和回答者（难度较大， SPI 在 编程导航的手写 RPC 项目中讲解过）

3）添加配置校验机制，在启动时检查配置的合法性

4）实现配置热更新，支持在不重启服务的情况下修改任务配置。

可以利用 Nacos 实现，在 编程导航的智能面试刷题平台项目 中讲过。